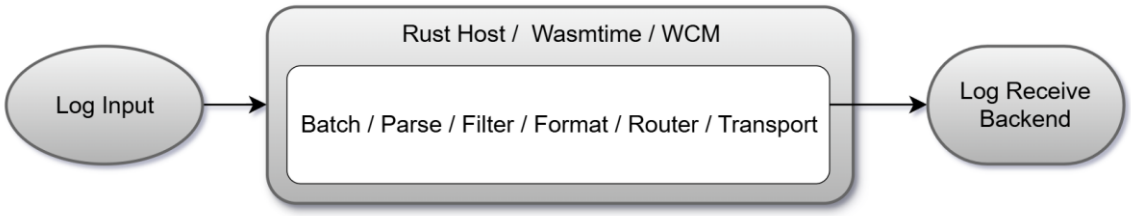
以 WebAssembly Component Model 實作日誌轉發框架

量化效能成本與可行性研究

摘要

本研究針對現有日誌轉發系統在擴充自訂邏輯時，常受**特定程式語言與產品介面限制、元件故障可能影響整體處理流程**，以及**缺乏存取權限制**等問題，提出一套**基於 WebAssembly Component Model(WCM)的元件化日誌轉發系統**。系統以 WIT 定義 Parser、Filter、Format 與 Transport 介面，使不同語言開發的元件可由 Rust 開發的 Host 統一載入及熱替換；並結合 Wasmtime 的記憶體限制、執行時間限制與能力導向介面，縮小無窮迴圈、記憶體超限及未授權外部存取的影響範圍。

實驗結果顯示，本系統可執行以 Rust、C、C#及 Go 開發的不同介面元件。在 1024 B、36 欄位的 JSON 日誌下，最大可持續吞吐量為 25,000 EPS，高於 Vector 與 Fluent Bit WASM，但低於 Fluentd 與 Fluent Bit Lua；在相同 5,000 EPS 下，本系統的 p99 延遲為 263.959 ms，低於其他受測配置。系統在 5、15 及 30 秒突發流量後皆能恢復穩態，且系統在一小時正式高負載觀察期間維持穩定；包含 180 秒暖機的完整試驗共正確送達 30,240,000 筆預期輸出，未發生資料錯誤、OOM 或重啟。結果顯示，WCM 可作為兼具跨語言擴充、故障隔離與可接受效能的日誌處理架構；惟其吞吐量仍不及部分成熟產品，仍有進一步改善空間。



圖一 系統架構概覽，中間為本系統的角色

研究問題

RQ1：WCM 與 WIT 能否降低元件對特定程式語言及宿主介面的依賴，並限制其資源、權限與故障影響範圍？

此問題探討 WCM 是否能改善傳統插件受限於特定語言，以及錯誤可能影響整體系統的問題。良好的跨語言支援與故障隔離，是提升系統擴充性與安全性的關鍵。

RQ2：相較於 Core Wasm 與現有日誌轉發產品，WCM 在效能與資源使用上具有哪些成本及取捨？

WCM 雖能提供結構化介面與元件隔離，但資料跨越元件邊界可能產生額外成本。

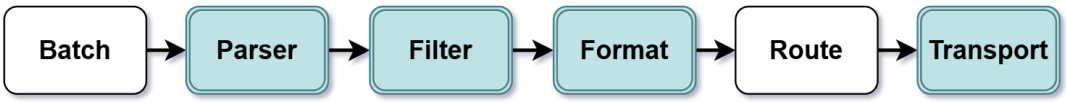
因此，本研究比較其吞吐量、延遲、CPU、記憶體及資料交換成本，以確認其實際可行性。

RQ3：WCM 日誌轉發系統在高負載、突發流量及長時間執行下，能否維持正確性與穩定性？

日誌系統必須持續處理大量且不穩定的流量，一旦發生資料遺失或無法恢復，可能影響後續監控與故障分析。因此，本研究驗證系統的資料正確性、資源穩定性及過載後恢復能力。

解決方法

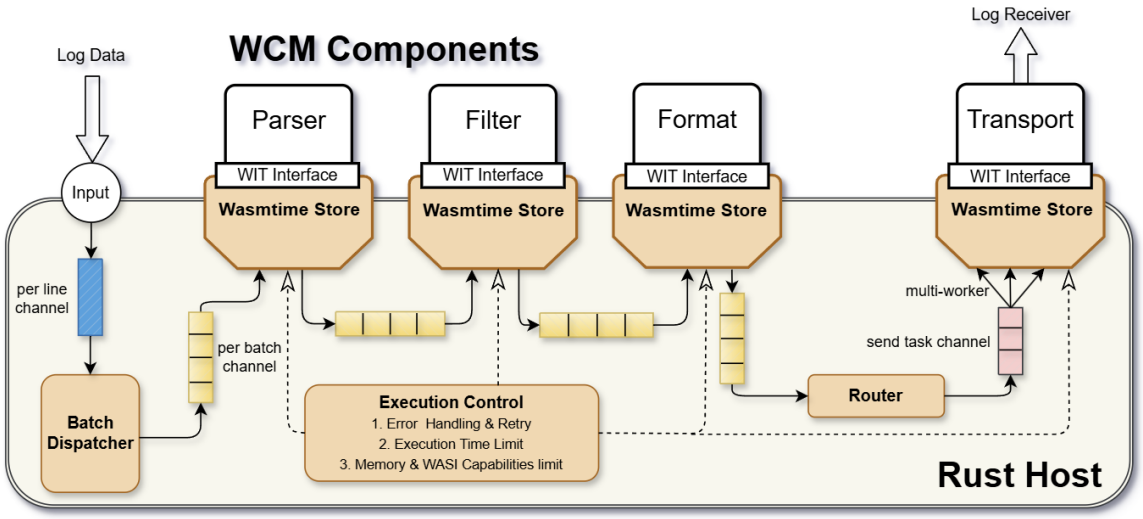
本研究提出一套基於 WebAssembly Component Model 的元件化日誌轉發系統，將日誌處理流程劃分為 Parser、Filter、Format 及 Transport 四個可替換元件，並以 WIT 定義共用資料型別與介面，使不同程式語言開發的元件皆能由 Rust Host 統一載入與呼叫。



圖二 資料處理流程，綠色代表可替換的元件

RQ1 的解決方法

本研究以 WIT 定義各階段的資料型別與呼叫介面，降低元件對來源語言及宿主實作的依賴。Rust Host 透過 Wasmtime 設定元件的記憶體與執行時間限制，並控制可匯入的外部能力；當元件發生錯誤時，由 Host 執行重試及失敗批次處置，以限制故障影響範圍。系統架構如圖三所示，各處理階段透過有界 Channel 傳遞批次資料；當下游處理速度不足時，反壓會傳回上游，限制資料持續累積。Router 則依目的端建立傳送任務，交由 Transport 元件送出。



圖三 本系統架構圖

RQ2 的解決方法

為了比較開發出來的系統與 Core WASM 的成本、效能差異，本研究使用與本系統相同的 Rust Host 再重新建立一套以 Core WASM 為呼叫元件的相同系統，與原系統只差在呼叫 WCM Component 或 WASM，Core WASM 使用的 ABI 則有 1. Length+Payload 2. JSON 兩種格式，各自代表高效能、高管理成本與易用但低效的兩個對照組，各自比較元件的呼叫成本。

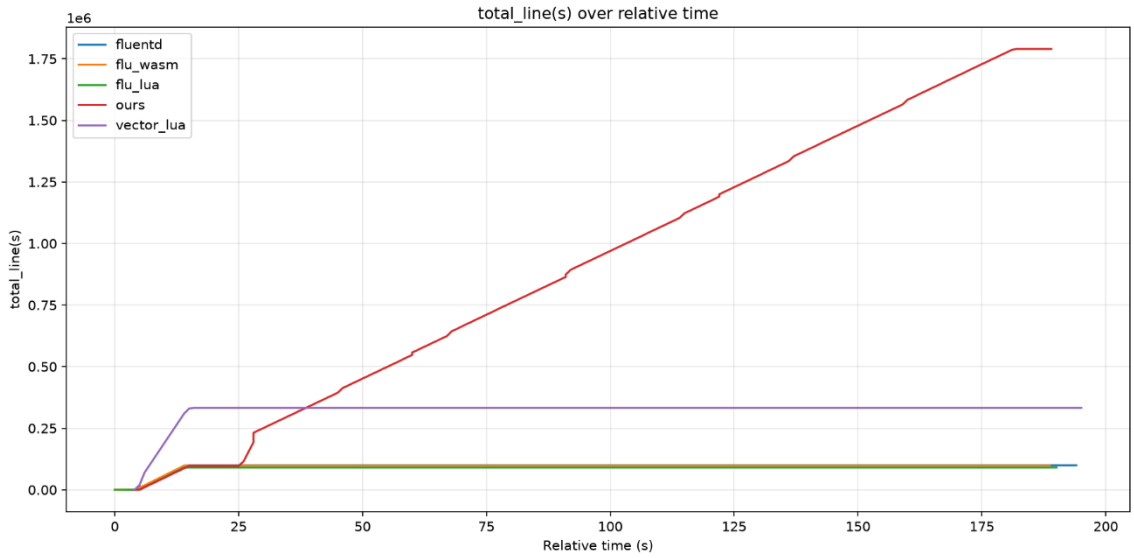
在市面產品比較方面，本研究選擇 Fluent Bit Lua、Fluent Bit WASM、Fluentd 及 Vector，在相同 CPU、記憶體、輸入資料與過濾規則下，比較不同大小 JSON 日誌的吞吐量、延遲、CPU 及記憶體使用量。由於各產品的批次與 flush 機制無法完全一致，因此結果用於分析整體系統的效能取捨，不將差異全部歸因於 WCM。

RQ3 的解決方法

本研究使用 1024 Bytes、36 欄位的 JSON 日誌進行正確性、突發流量及長時間穩定性實驗。長時間實驗將系統維持在最高穩定吞吐量的 80%，連續執行一小時，觀察資料遺失、重複、內容錯誤、延遲、CPU、記憶體、OOM 及重啟情形。突發流量實驗則將流量提高至最高穩定吞吐量的 150%，分別持續 5、15 及 30 秒，再降低至基準流量，量測系統的延遲、backlog、恢復時間及資料正確性，並與 Fluentd、Vector 及 Fluent Bit WASM 進行比較。

實驗結果與比較

RQ1 實驗結果

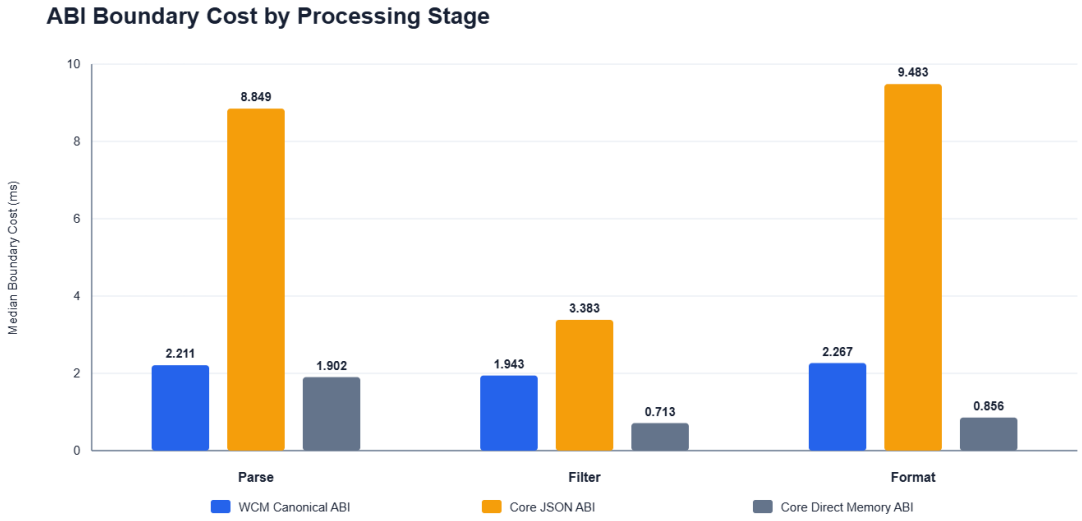


圖四 邏輯中產生無窮迴圈對不同產品造成的影響

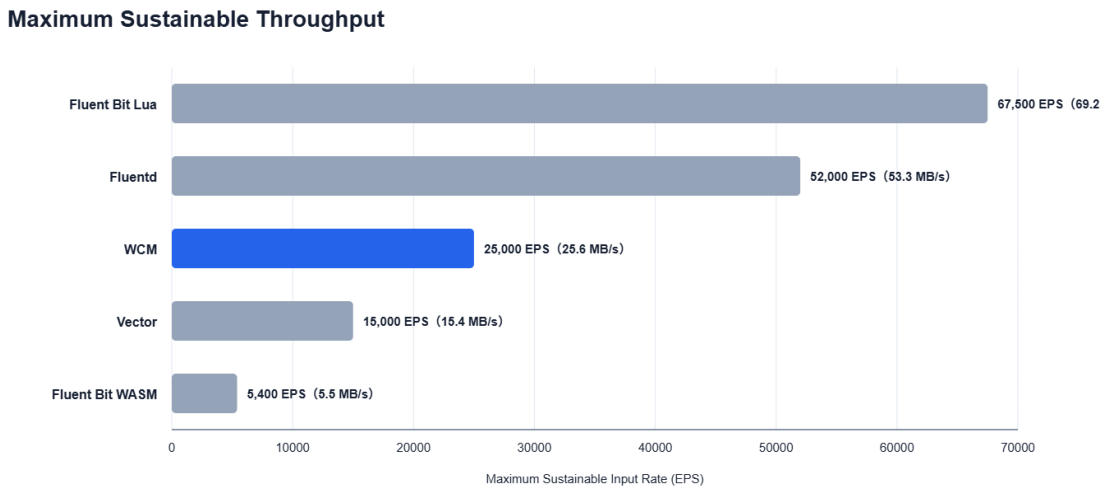
本研究也實際使用 Rust、C、C#、GO 四種語言實作不同階段的元件，並且能夠正常完成轉發，顯示 WIT 作為介面描述語言搭配 WCM 能夠降低語言的耦合，且 Host 也能針對元件的記憶體上限做出限制，使元件在複製到上限時進行產生錯誤，限制記憶體無限增長。

圖四為在插入自訂邏輯中，設定約在日誌開始輸入後的第 15 秒時，讀取到輸入的日誌的特定欄位時並產生無窮迴圈，測試各產品對於此情況下的應對方式，結果顯示，Fluent Bit、Fluentd、Vector 三個產品的插件架構均無法正常防止上述情況，在 15 秒後無法繼續轉發日誌，本系統透過對插件的時間上限，能夠重試並將持續失敗、有問題的日誌寫入檔案，不影響後續正常日誌的轉發。

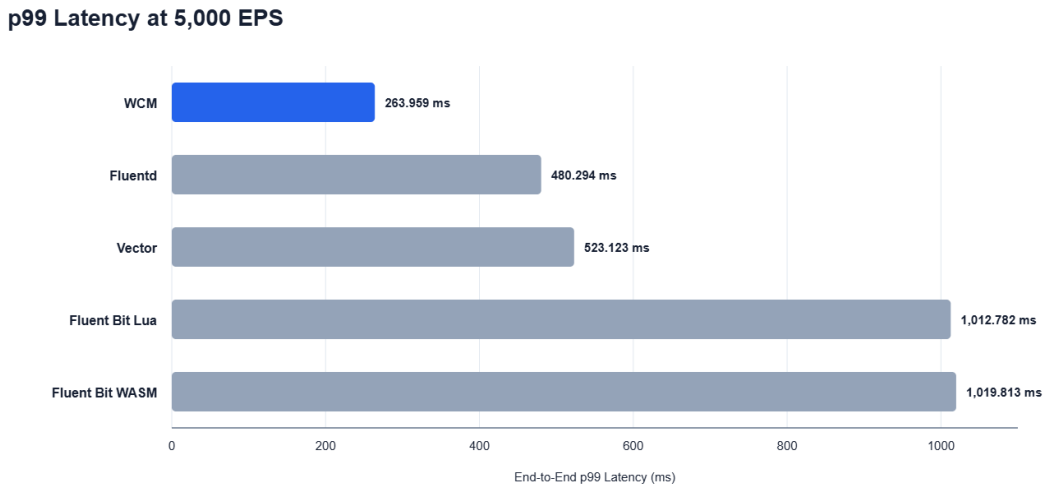
RQ2 實驗結果



圖五 比較 WCM 與 Core WASM 的不同 ABI 邊界成本差異



圖六 比較本系統與不同產品的最大吞吐量



圖七 不同系統在相同工作量與資源限制下的整體端到端延遲。

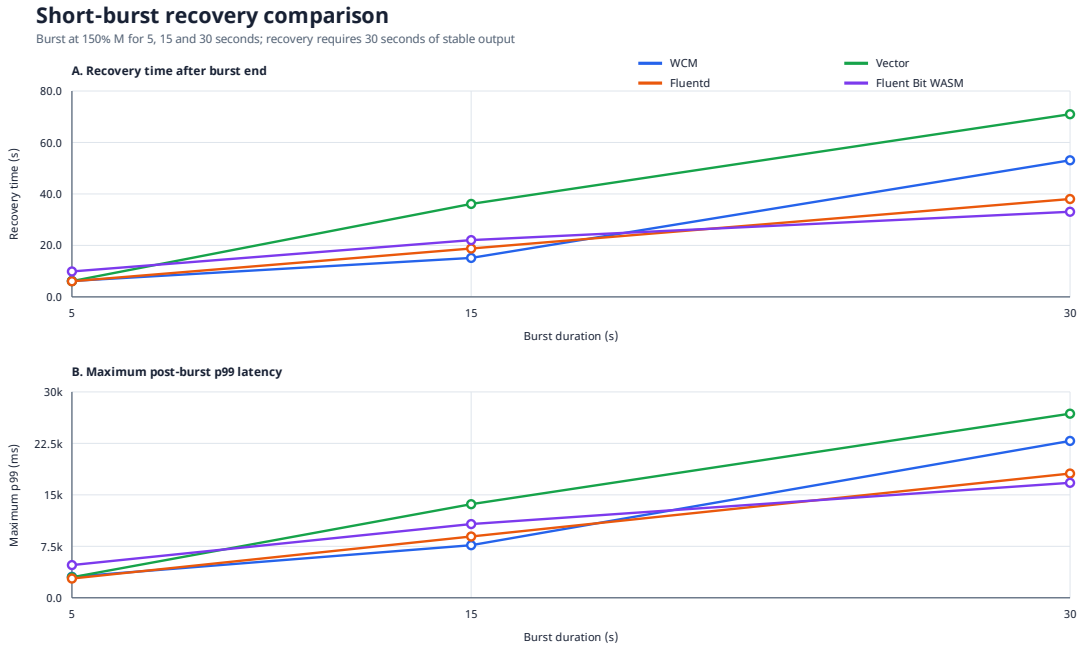
圖五的實驗以 1,024 筆、約 1 MiB 的資料，比較 WCM、JSON ABI 與 Direct Memory ABI 在 Parse、Filter、Format 階段的資料輸入與輸出成本，不包含元件內部的業務邏輯時間。結果顯示，WCM 的邊界成本介於另外兩種 ABI 之間。WCM 明顯優於 JSON ABI，但仍高於需要手動管理記憶體的 Direct Memory ABI。這表示 WCM 能以部分執行成本換取型別安全、語言中立及較低的介面維護成本。

圖六的實驗在 1 CPU、512 MiB 記憶體下，使用 1,024 Bytes、36 欄位的 JSON 日誌與相同過濾條件，測量各系統可穩定處理的最高 EPS。WCM 的最大可持續吞吐量為 25,000 EPS，高於 Vector 的 15,000 EPS 與 Fluent Bit WASM 的 5,400 EPS，但低於 Fluentd 的 52,000 EPS 及 Fluent Bit Lua 的 67,500 EPS。結果顯示，本系統雖具有元件隔離與跨語言能力，仍能維持具有競爭力的吞吐量，但在峰值吞吐量方面尚有優化空間。

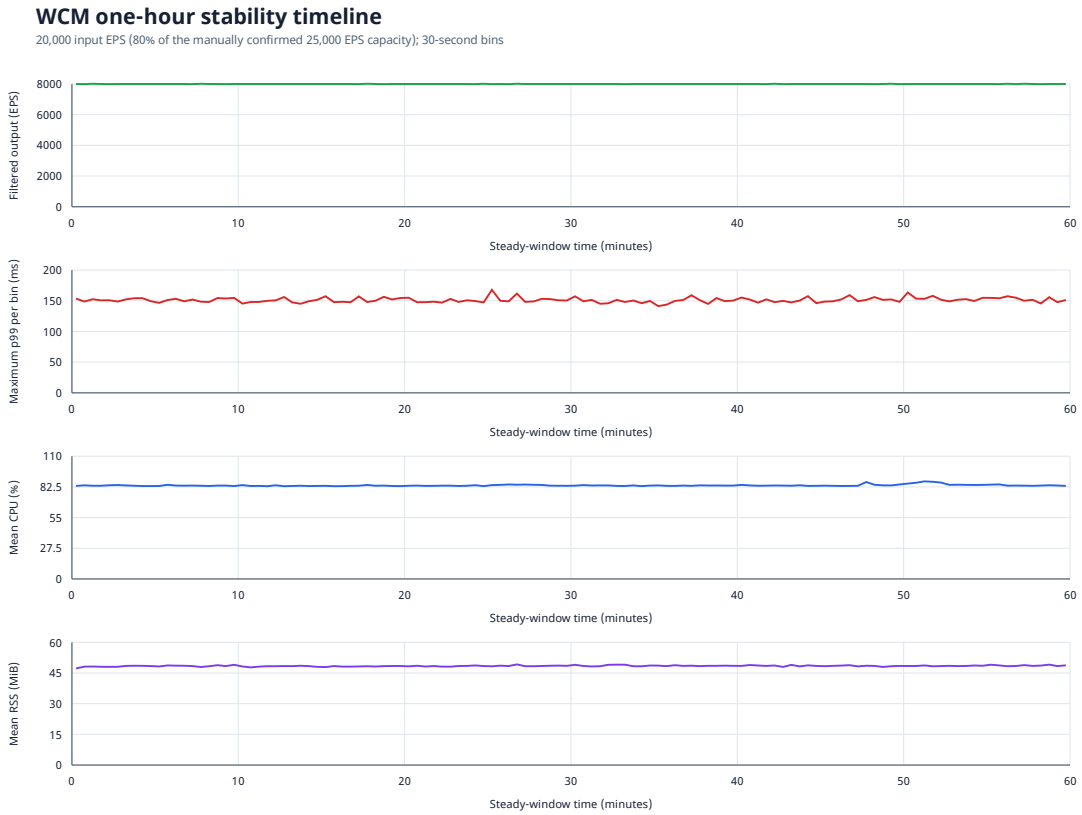
圖七比較各系統在相 5,000 EPS 負載下的 p99 端到端延遲。實驗使用每筆 1,024 Bytes、36 個欄位的 JSON 日誌，與相同的過濾邏輯。所有受測系統均限制為 1 CPU、512 MiB 記憶體及單一輸出 worker。每組實驗執行 180 秒，取第 30 至 150 秒作為穩態量測區間。延遲定義為日誌的排程產生時間，至接收端完整讀取 HTTP request body 的時間差。各系統皆設定 1 秒 flush；WCM、Fluentd 及 Vector 使用約 1 MiB 的批次上限，Fluent Bit 則受產品機制限制，使用原生約 2 MiB 的 soft chunk。

在相同 5,000 EPS 負載下，WCM 的 p99 端到端延遲為 263.959 ms，低於 Fluentd、Vector、Fluent Bit Lua 及 Fluent Bit WASM，顯示本系統在共同負載下具有較佳的尾端延遲表現。然而，由於各產品的批次形成位置與 flush 機制不同，本結果應視為整體系統效能的比較，而非 WCM 執行成本的單獨比較。

RQ3 實驗結果



圖八 突發恢復時間與突發後最嚴重的尾延遲



圖九 本系統的穩定性實驗結果(平均過濾後吞吐、p99 延遲、平均記憶體與 CPU)

RQ3 主要為本系統的正確性、穩定性實驗。圖八代表經過 5、15、30 秒的 150%最

大吞吐作為突發流量，測量突發後的恢復時間與 Post-burst p99，恢復時間自突發結束起算，定義為第一個符合輸出率、延遲及積壓門檻之連續 30 秒區間的起點。Post-burst p99 則取突發結束後觀察期間內，各秒 p99 延遲的最大值。結果顯示在 5 秒突發下，WCM 的恢復時間及 Post-burst p99 與 Fluentd、Vector 接近；在 15 秒突發下，兩項指標均為受測系統中最低。然而，30 秒突發造成較明顯的資料積壓，使恢復時間與 Post-burst p99 增加，表現僅優於 Vector。

一小時穩定性實驗以 20,000 EPS（容量測試所得最大吞吐量的 80%）持續輸入日誌，觀測過濾後輸出速率、p99 延遲、記憶體及 CPU 使用量。如圖九所示，輸出速率維持穩定，延遲與資源使用量未呈現持續上升趨勢；資料驗證亦未發現遺失、重複或內容錯誤，且未發生 OOM 或重啟。結果支持本系統在此負載下的一小時運作穩定性。

合法資料情境下未觀察到資料錯誤；但格式錯誤注入實驗顯示，單筆無效 JSON 可能導致同批合法日誌一併被捨棄，顯示目前錯誤隔離仍以批次為主。

結論與未來工作

本研究提出並實作一套以 WCM 為基礎的元件化日誌轉發系統，驗證以 WIT 統一介面、由 Rust Host 管理元件執行的架構具有可行性。本研究的主要結論是：跨語言擴充與故障隔離可以整合於同一套日誌處理流程，但其效果來自 WCM 與 Host 控制機制的共同設計。WIT 降低了元件對來源語言及宿主實作的耦合；Host 則負責資源限制、能力授權與錯誤處置，使自訂邏輯的故障影響範圍更可控。

效能實驗顯示，WCM 的邊界成本介於 Direct Memory ABI 與 JSON ABI 之間，反映標準化介面與低階效能之間的取捨。在產品比較中，本系統的吞吐量高於 Vector 及 Fluent Bit WASM，並在相同 5,000 EPS 下取得最低的 p99 延遲，但吞吐量仍低於 Fluentd 及 Fluent Bit Lua。這些結果支持 WCM 作為重視擴充性與隔離能力之日誌框架的可行選擇，並不表示其在所有工作負載下均優於既有產品。

穩定性實驗顯示，有界佇列與反壓設計能協助系統承受短時間過載，並在突發結束後恢復；一小時高負載測試亦未觀察到資料錯誤、OOM 或重啟。然而，較長突發仍會造成明顯排隊延遲，格式錯誤也可能影響整個批次，顯示執行隔離尚不足以保證逐筆資料的完整性。

未來將優先改善逐筆錯誤隔離、降低跨元件資料交換成本，並調整批次與佇列策略，以提升吞吐量及過載恢復能力。同時增加實驗重複次數與長時間測試，進一步確認本架構在不同負載下的適用範圍。